

기술 문서

프로젝트명 : UFPS



깃허브

<https://github.com/crescendo103/UFPS.git>



실행영상

https://youtu.be/pb_JKvnfxuQ

목차

- 프로젝트 소개
- 게임 소개
- 주요 클래스 구조
- 핵심 시스템
- 트러블 슈팅

본 문서에서는 코드 위주로 작성되었으며, 전체적인 내용은 영상을 통해 확인해 주시기 바랍니다.

프로젝트 소개



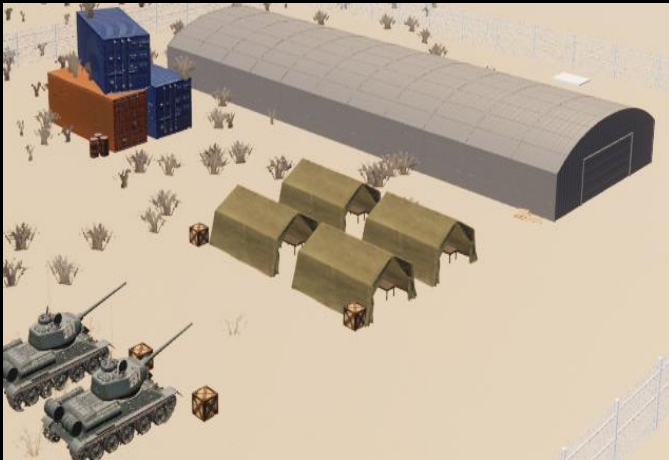
-로그인 화면-

- 게임 장르
1 vs 1 fps
- 개발 인원
1인 개인 제작
- 개발 기간
2026.01 ~ 2026.06
- 개발 환경
Unreal 5.4, VisualStudio2022

언리얼 엔진을 처음 공부하면서 학부과정에 배운 소켓 프로그래밍을 적용하여 만든 프로젝트입니다.

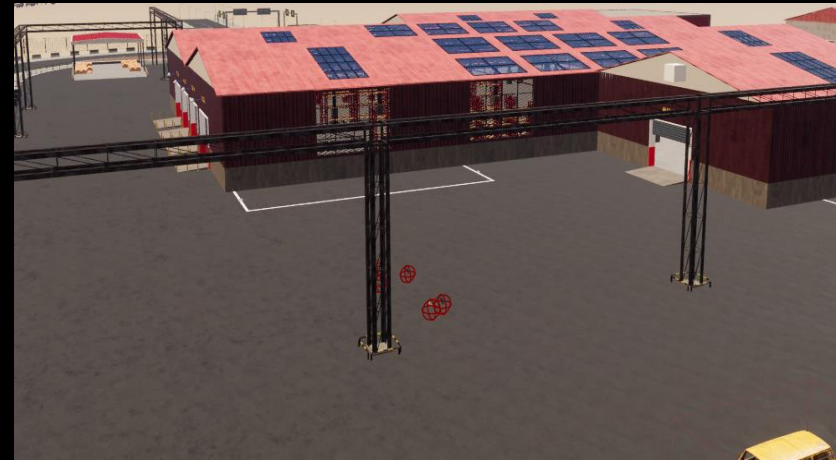
게임 소개

- 배틀그라운드류 배틀로얄 슈팅 게임으로, 낙하 후 아이템을 파밍하고 전투를 벌여 최후의 생존자를 가리기



시작 집결지

헬기로 수송

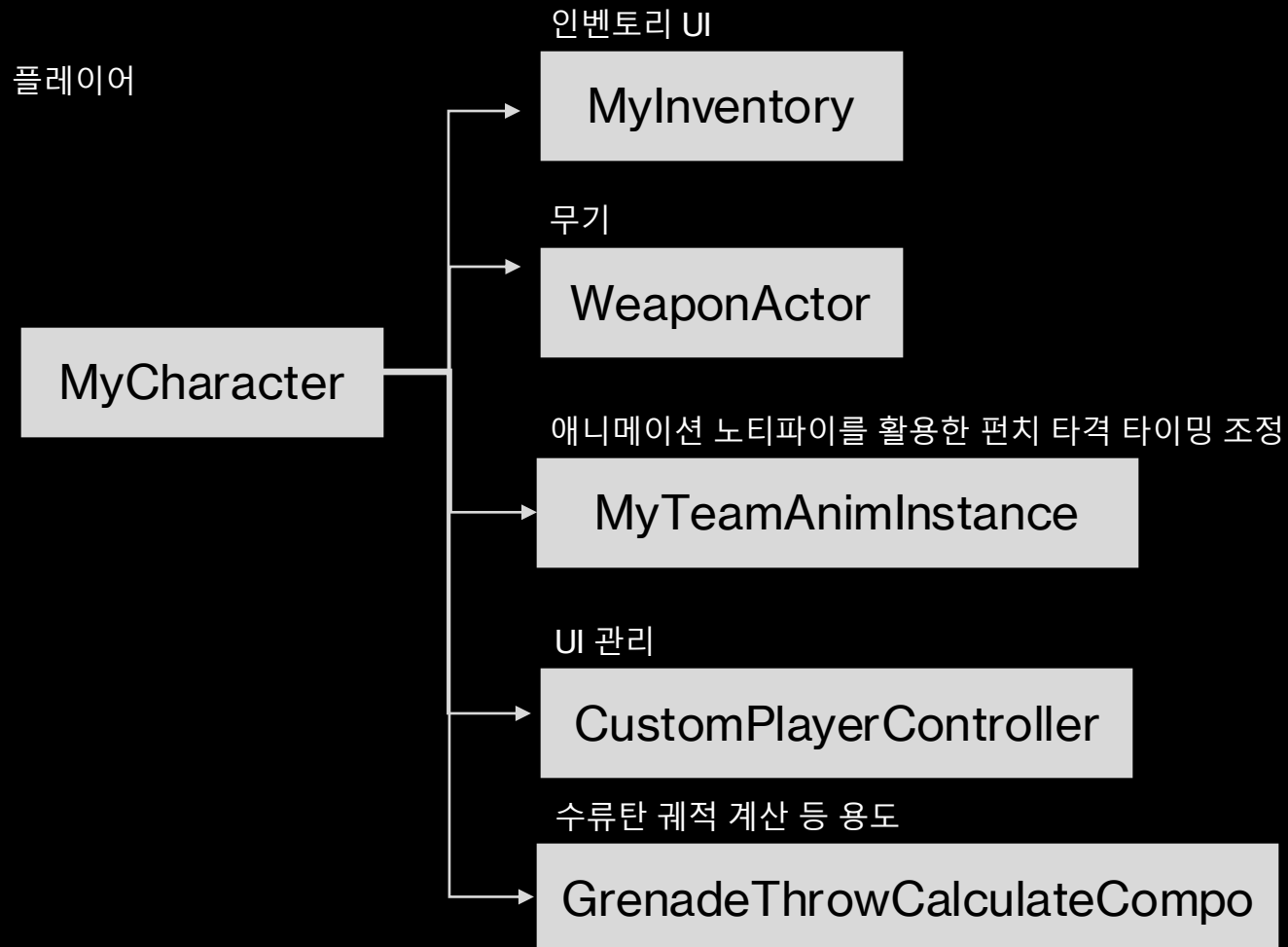


배틀 그라운드



아이템

주요 클래스 구조



주요 클래스 구조

무기액터



아이템 하이라이트용 인터페이스

HighlightInterface

무기 사용 로직

WeaponUseComponent

무기 사용, 파괴 사운드 및 파티클

ItemEffectComponent

WeaponActor

인벤토리



인벤토리 UI

MyInventory

HavelItemBox

가지고 있는 아이템 UI

WeaponBox

보조무기 UI

WeaponBoxPlayer

주무기 UI

TrashBorder

아이템 버리는 UI

핵심 시스템 - 전투

플레이어가 총기를 사용하게 되면

UWeaponUseComponent의 UseWeapon()과
SendBulletPacket()을 호출하게 됩니다.

이때 UseWeapon()는 본인 클라이언트에 총알을
그리는 역할을 하고

SendBulletPacket()는 총기 사용 주체 이외의
클라에게 총알을 그리라는 패킷을 서버로 보내게
됩니다.

```
void AWeaponActor::SpawnItem(FVector startPos, FVector startRot)
{
    if (SpawnComponent)
    {
        FVector startedPos = Mesh->GetSocketLocation(TEXT("SpawnPoint"));
        SpawnComponent->UseWeapon(Owner, startPos, startRot);
        SpawnComponent->SendBulletPacket(startPos, startRot); //서버 전송
    }
}
```

```
void URifleUseComponent::UseWeapon(AMyCharacter* Owner, FVector StartPos, FVector Dir)
{
    FActorSpawnParameters SpawnParams;
    SpawnParams.Owner = Owner; // 스폰 주체
    SpawnParams.Instigator = Owner; // 총알이 누구로부터 나온건지
    if (BulletClass) {
        ABullet* Bullet = GetWorld()->SpawnActor<ABullet>(BulletClass, StartPos, Dir.Rotation(), SpawnParams);
        Bullet->Throw(Dir, 3000);
    }
}
```

```
void URifleUseComponent::SendBulletPacket(FVector startPos, FVector dir)
{
    UMyServer* MyServer =
        GetWorld()->GetGameInstance()->GetSubsystem<UMyServer>();
    if (MyServer) {
        FVector StartPos = startPos;
        FVector Dir = dir;
        FServerBullet Bullet;
        Bullet.Header.Type = (int32)EPacketType::Bullet;
        Bullet.Header.Size = sizeof(FServerBullet);
        Bullet.BulletId = -1; //쓰레기값
        Bullet.CharacterId = -1; //쓰레기값
        Bullet.X = StartPos.X; //100.f;
        Bullet.Y = StartPos.Y; //200.f;
        Bullet.Z = StartPos.Z; //300.f;
        Bullet.DirX = Dir.X; //1.f;
        Bullet.DirY = Dir.Y; //0.f;
        Bullet.DirZ = Dir.Z; //0.f;
        Bullet.Speed = 1200.f;
        Bullet.Sendtime = FPlatformTime::Seconds();
        Bullet.flag = false;
        MyServer->Shotoccurred(Bullet);
    }
}
```

핵심 시스템 - 전투

이와 다르게 수류탄은 플레이어에

수류탄이 날아갈 각도를 Projectile로

계산하는 UGrenadeThrowCalculateCompo

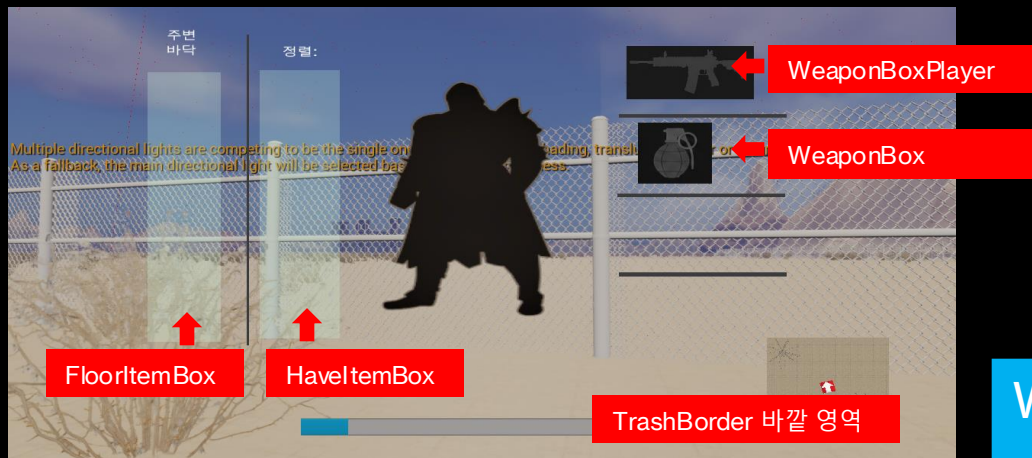
컴포넌트가 존재한다. 수류탄이 발사 되었을때 이 값을 토대로 클라에서 수류탄을 그리고 서버로 요청을 보낸다.

```
if (isSubWeaponActive)
{
    if(SubItemData.SubItemClass && SubItemData.Id!=0)//수류탄은 여기서 발사
    {
        FTransform SocketTransform = GetMesh()->GetSocketTransform("RightHandPinky4Socket", RTS_World);
        FActorSpawnParameters SpawnParams;//스폰할 액터의 옵션 지정
        SpawnParams.Owner = this;//소유자를 캐릭터로
        SpawnParams.Instigator = GetInstigator();//이 무기를 스폰한 주체가 누군지
        SubItem = GetWorld()->SpawnActor<AMyGrenade>(SubItemData.SubItemClass, SocketTransform, SpawnParams);
        //SubItem = GetWorld()->SpawnActor<AMyGrenade>(GrenadeClass, SocketTransform, SpawnParams);

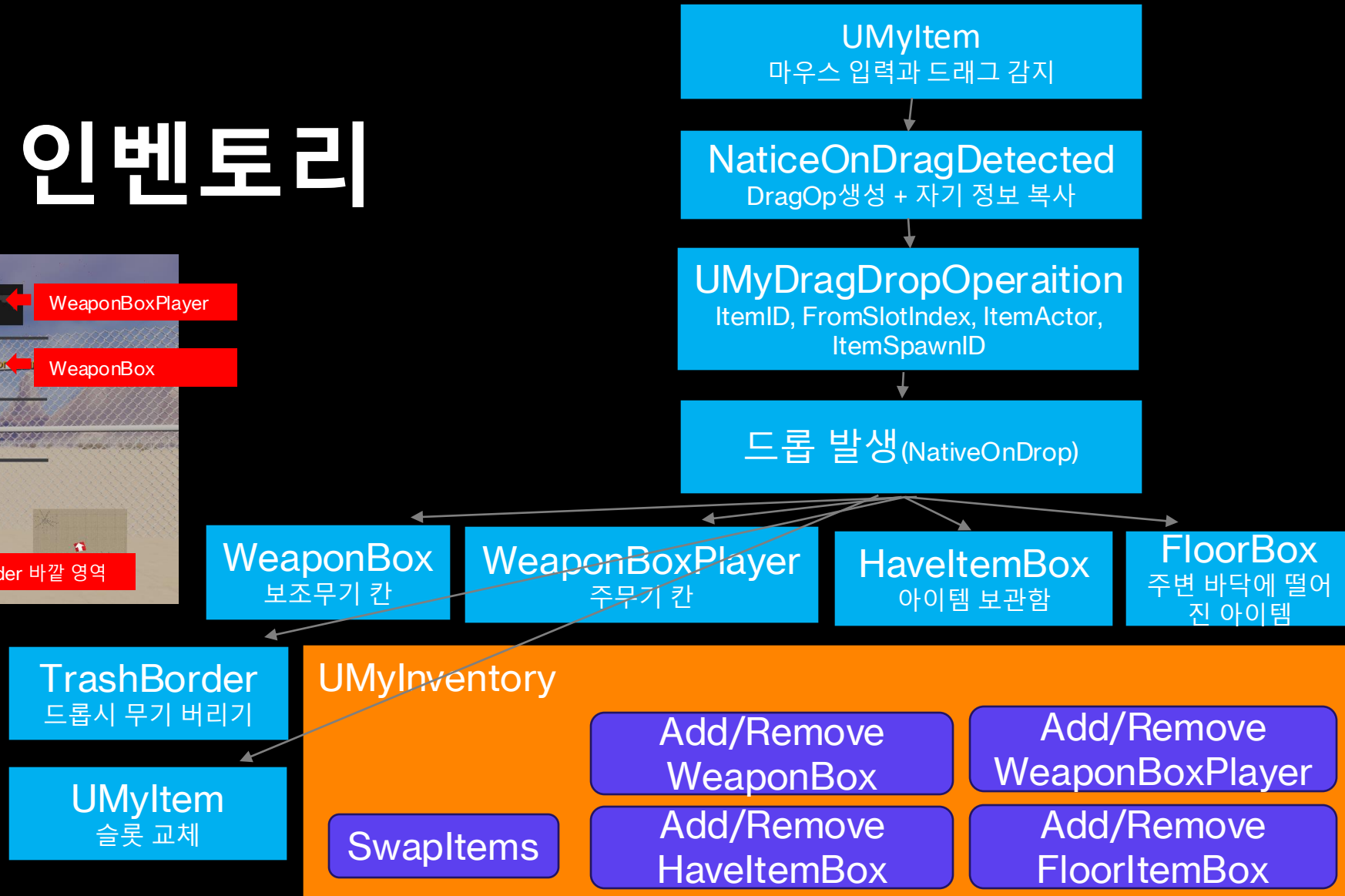
        AMyGrenade* Grenade = Cast<AMyGrenade>(SubItem);
        FVector Direction = FVector::ForwardVector;
        if (GrenadeCalComponent)
        {
            Direction = GrenadeCalComponent->GetFinalThrowDirection();
        }
        //던지기
        if (Grenade)
        {
            Grenade->Throw(Direction, 1000.f);
        }

        FGrenadePacket packet;
        packet.Header.Type = (int32)EPacketType::Grenade;
        packet.Header.Size = sizeof(FGrenadePacket);
        packet.GrenadeId = SubItemData.Id;//수류탄 ID 아마 안쓸듯
        packet.CharacterId = -2;//trash
        packet.X = SocketTransform.GetLocation().X;
        packet.Y = SocketTransform.GetLocation().Y;
        packet.Z = SocketTransform.GetLocation().Z;
        packet.DirX = Direction.X;
        packet.DirY = Direction.Y;
        packet.DirZ = Direction.Z;
        packet.Power = 1000;
        MyServer->CreateGrenade(packet);
    }
}
```


핵심 시스템 - 인벤토리



-인벤토리 구조도-



```
UCLASS()
파생된 블루프린트 클래스 0개
class FPS_API UMyDragDropOperation : public UDragDropOperation
{
    GENERATED_BODY()
public:
    UPROPERTY()
    0 Blueprints에서 변경됨
    int32 ItemID; // ItemDataTable의 ID, 같은 종류 아이템이면 같음

    UPROPERTY()
    0 Blueprints에서 변경됨
    int32 FromSlotIndex; // inventory 에서 아이템 슬롯 번호

    UPROPERTY()
    0 Blueprints에서 변경됨
    AActor* ItemActor=nullptr; // weaponActor
    UPROPERTY()
    0 Blueprints에서 변경됨
    int32 ItemSpawnID; // 월드의 고유 ID, 같은 종류 아이템이라도 다름
};
```

핵심 시스템 - 인벤토리

인벤토리 드래그앤드롭 핵심 흐름

드래그 시작 (UMyItem)

플레이어가 슬롯을 마우스로 누르면, UMyItem이 마우스 입력을 받아 드래그 후보로 등록한다. 일정 거리 이상 끌리면 NativeOnDragDetected가 호출되어, 이 슬롯이 들고 있던 아이템 정보(ID, 슬롯 번호, 액터, 스폰ID)를 그대로 복사해 UMyDragDropOperation이라는 데이터 캡슐을 만든다. 이 캡슐이 마우스를 따라다니며 드래그가 어디서 끝나는지를 결정한다.

드롭 발생, 4가지 경로로 분기

마우스를 놓으면 NativeOnDrop이 호출되고, 어디에 떨어졌는지에 따라 처리 방식이 갈린다.

- **WeaponBox** (보조무기 칸) – 수류탄 등 보조무기를 장착하는 칸. 화면용 무기 액터를 스폰해 부착하지만, 실제 동작 로직은 별도 액터에서 처리한다.
- **WeaponBoxPlayer** (주무기 칸) – 유일하게 실제 캐릭터 손에 무기를 부착(WeaponAttach)하고 현재 무기 상태를 갱신하는 칸. 단순 UI 슬롯이 아니라 "지금 들고 있는 무기"를 결정한다.
- **HavelItemBox** (아이템 보관함) – 획득한 아이템을 인벤토리에 보관. 보조무기 칸에서 옮겨온 경우 기존 장착을 해제한다.
- **FloorBox** (주변 바닥 아이템) – 플레이어 근처 바닥에 떨어진 아이템을 UI에 표시만 하는 칸. 실제 슬롯 등록 없이 보여주기 전용이다.

같은 슬롯끼리 드롭하면 (SwapItems)

인벤토리 슬롯 위에 드롭하면 위 네 칸으로 분기하지 않고, SwapItems가 두 슬롯의 데이터(아이템ID, 액터, 위젯, 스폰ID)를 통째로 교환한다.

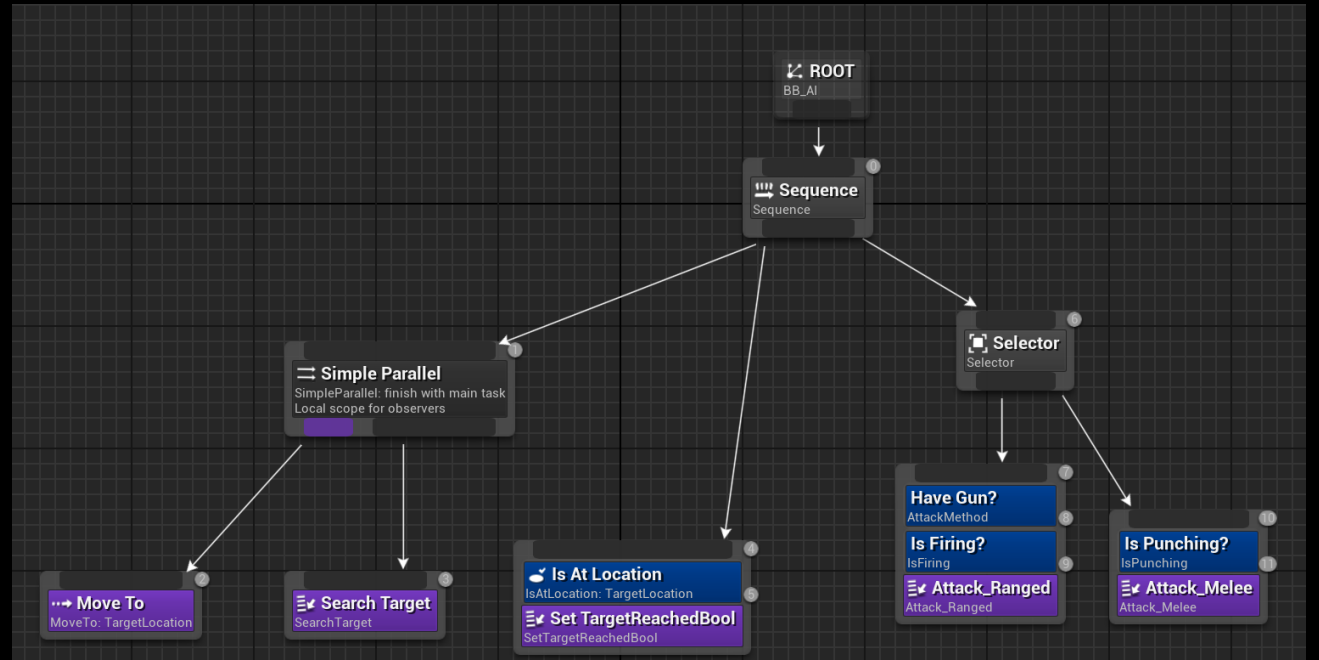
모든 경로가 도착하는 곳: UMyInventory

네 가지 드롭 대상 모두 결국 UMyInventory의 Add/Remove 계열 함수를 호출해 Slots(슬롯 번호 → 아이템 정보 맵)를 갱신한다.

예외: TrashBorder

다른 드롭 대상과 달리, 드롭된 아이템을 인벤토리에 등록하는 게 아니라 곧바로 버리는 처리를 한다. UMyItem을 그대로 재사용해 "슬롯 교체"라는 개념 없이 제거만 수행한다.

핵심 시스템 - AI



전체적인 흐름을 적어보면

Move to 는 서버가 지정해준 좌표로 AI가 이동합니다

SearchTarget는 AI앞에있는 BoxCollision에 플레이어 또는 아이템이 오버랩 될 경우 플레이어를 공격하거나 아이템을 획득하는데 사용됩니다.

Set TargetReachedBool는 목표 위치에 도달했을 경우(Partrol) 서버로 새로운 위치를 보내달라는 패킷을 보내게 됩니다.

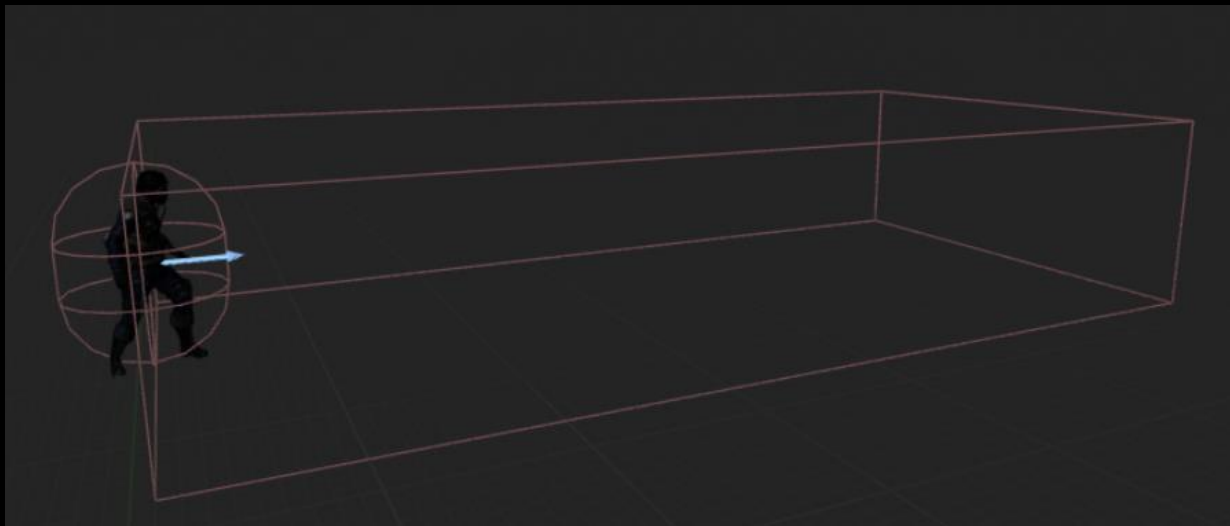
Have Gun? 데코레이터는 총을 가지고 있을경우 총을 발사하도록 Attack_Ranged Task를 실행하도록 하였고 아닐경우 맨손공격을 하도록합니다

(총기에 대한 부분은 구현이 되지 않았음)

핵심 시스템 - AI

SearchTarget는

AI앞에있는 BoxCollision에 플레이어 를 우선
공격하고 없다면 아이템 획득을 없다면 기존
Patrol를 유지합니다.



```
// Player 우선
if (Actor->IsA(AMyCharacter::StaticClass()))
{
    FVector pos = Actor->GetActorLocation();
    Enemy->RequestNewPos(pos.X, pos.Y, pos.Z, false, true);
    UE_LOG(LogTemp, Warning, TEXT("BTTask: Player detected"));

    return EBTNodeResult::Succeeded;
}

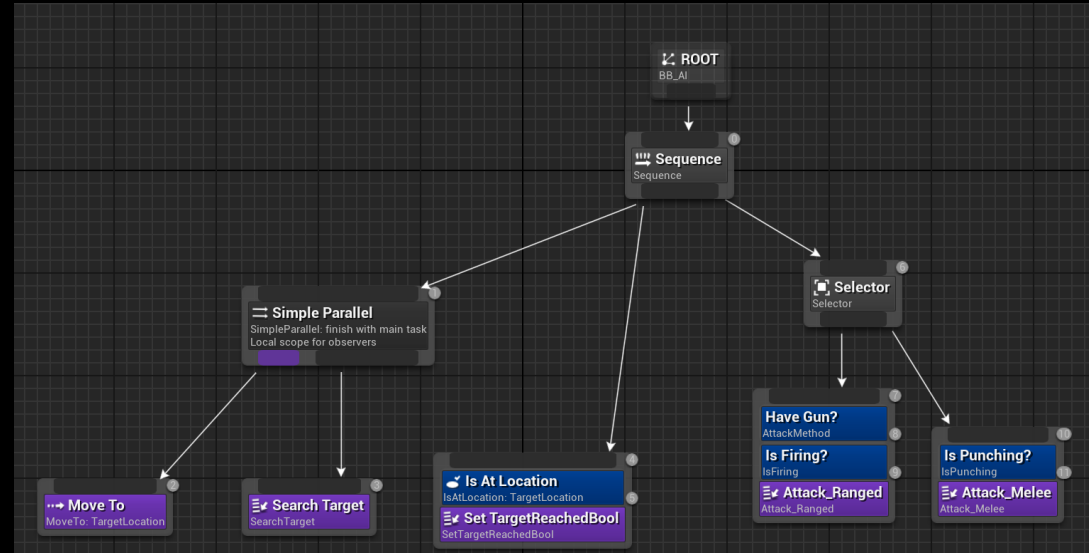
// Item은 후보로만 저장
if (!FoundItem && Actor->IsA(AWeaponActor::StaticClass()))
{
    FoundItem = Actor;
}

// Player 없고 Item만 있을 때
if (FoundItem)
{
    FVector pos = FoundItem->GetActorLocation();
    Enemy->RequestNewPos(pos.X, pos.Y, pos.Z, false, true);

    UE_LOG(LogTemp, Warning, TEXT("BTTask: Item detected"));
    return EBTNodeResult::Succeeded;
}

return EBTNodeResult::Succeeded;
```

핵심 시스템 - AI



Set
TargetReachedBool는 목표
위치에 도달했을 경우(Patrol)
서버로 새로운 위치를
보내달라는 패킷을 보내게
됩니다.

```

EBTNodeResult::Type UBT_SetTargetReachedBool::ExecuteTask(
    UBehaviorTreeComponent& OwnerComp,
    uint8* NodeMemory)
{
    AAIController* AI = OwnerComp.GetAIOwner();
    if (!AI) return EBTNodeResult::Failed;

    UBlackboardComponent* BB =
        OwnerComp.GetBlackboardComponent();
    if (!BB) return EBTNodeResult::Failed;

    AAIEnemy* Enemy = Cast<AAIEnemy>(AI->GetPawn());
    if (!Enemy) return EBTNodeResult::Failed;

    Enemy->RequestNewPos(0,0,0,true,false);

    return EBTNodeResult::Succeeded;
}

```

```

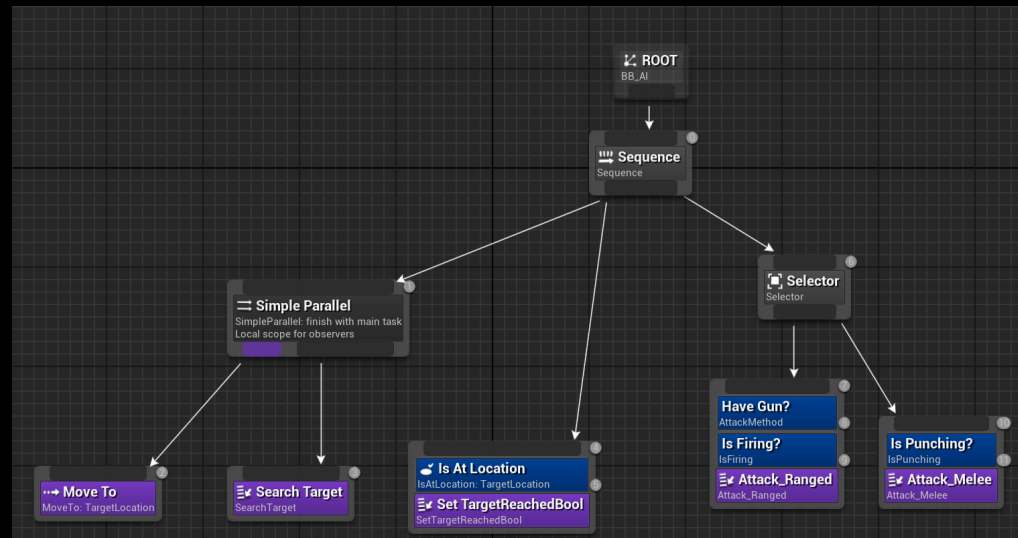
void AAIEnemy::RequestNewPos(float x, float y, float z, bool Jump, bool Fire)
{
    UE_LOG(LogTemp, Warning, TEXT("AAIEnemy BB >> RequestNewPos"));
    float Speed = GetCharacterMovement()->Velocity.Size();
    FVector MoveDir = GetVelocity().GetSafeNormal();//실제 이동 방향
    FVector ForwardDir = GetActorForwardVector();//바라보는방향
    FVector Location = GetCharacterMovement()->GetActorLocation();//위치

    FCharacterPacket man;
    man.Header.Type = (int32)EPacketType::Character;
    man.Header.Size = sizeof(FCharacterPacket);
    man.CharacterId = AiId;//AI식별
    man.X = Location.X;//100.f;
    man.Y = Location.Y;//200.f;
    man.Z = Location.Z;//300.f;
    man.DirX = x;
    man.DirY = y;
    man.DirZ = z;
    man.Speed = Speed;
    man.Sendtime = FPlatformTime::Seconds();
    man.IsJump = Jump;//아마 세워치 보내라는 flag
    man.IsFire = Fire;//이거는 죽음 여부를 나타내는 듯
    man.IsDeath = false;
    man.IsHeal = false;
    man.WeaponType = EWeaponType::Melee;
    man.AimActive = false;

    if (MyServer) {
        MyServer->MoveAI(man);
    }
    BB->SetValueAsBool("TargetReached", false);
}

```

핵심 시스템 - AI



Attack_Melee task는 AI Pawn에서 충돌 구체를 만들도록 생성합니다

```

EBTNodeResult::Type UBT_Attack_Melee::ExecuteTask(UBehavior
{
    AAIController* AIcon = OwnerComp.GetAIOwner();
    AAIEnergy* Enemy = Cast<AAIEnergy>(AIcon->GetPawn());

    if (!Enemy)
        return EBTNodeResult::Failed;
    Enemy->AttackMelee(); // 애니 + 데미지

    return EBTNodeResult::Succeeded;
}
  
```



```

void AAIEnergy::AttackMelee()
{
    FVector Start = GetActorLocation();
    FVector Forward = GetActorForwardVector();
    float MeleeDamage = 10;
    float MeleeRange = 10;
    float MeleeRadius = 100;

    // 공격 위치 (캐릭터 앞)
    FVector AttackCenter = Start + Forward * MeleeRange;

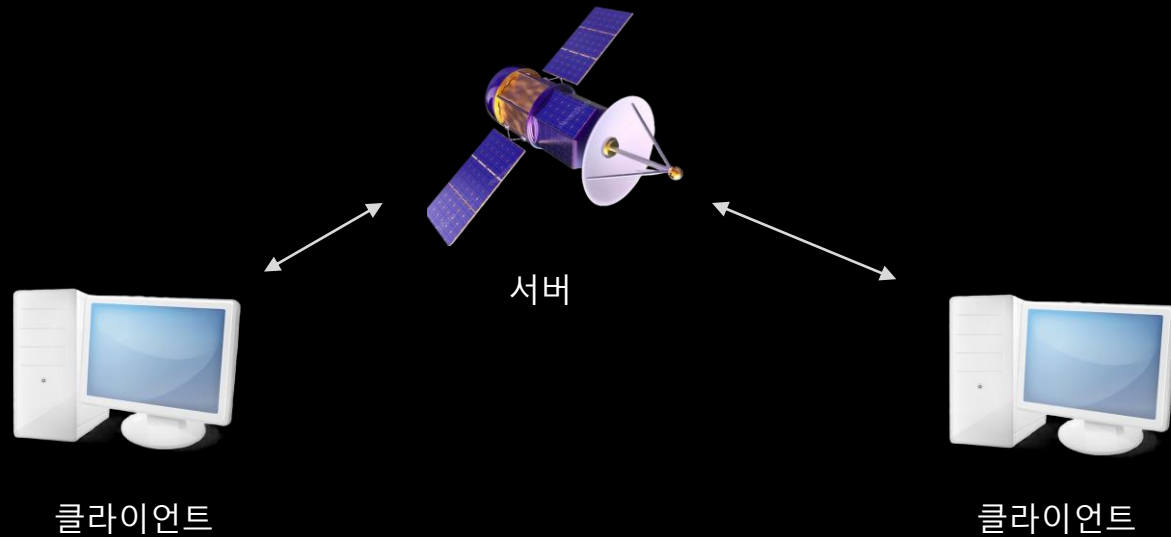
    // 충돌 설정
    TArray<FHitResult> HitResults;
    FCollisionShape Sphere = FCollisionShape::MakeSphere(MeleeRadius);

    FCollisionQueryParams Params;
    Params.AddIgnoredActor(this); // 자기 자신 제외

    bool bHit = GetWorld()->SweepMultiByChannel(
        HitResults,
        AttackCenter,
        AttackCenter, // Start == End → 고정 Sphere
        FQuat::Identity,
        ECC_GameTraceChannel2,
        //ECC_Pawn, // 캐릭터만 맞추기
        Sphere,
        Params
    );

    // 디버그 구체
    DrawDebugSphere(
        GetWorld(),
  
```

핵심 시스템 - 서버와 통신(클라이언트)



`++ MyServer.cpp`

MyServer는 패킷을 보내고 처리하는 클래스입니다.

`++ ClientTrd.cpp`

ClientTrd는 비동기 스레드로 서버로부터 오는 패킷을 받아 큐에 넣습니다.

핵심 시스템 - 서버와 통신

```
// IP, Port 정보 입력
SOCKADDR_IN stServerAddr;
stServerAddr.sin_family = AF_INET;
stServerAddr.sin_port = htons(6000);
stServerAddr.sin_addr.s_addr = inet_addr("127.0.0.1");//내 pc ip
```

클라와 서버간 통신은 루프백 주소
localhost에서만 이루어졌습니다.

클라이언트는 서버로 값을 보내거나 처리하고



서버로부터 받은 패킷을 ClientTrd클래스를
수신용 쓰레드로 생성하여 비동기적으로 큐에 넣습니다.



```
void UMyServer::MoveVehicle(FVehiclePacket packet)
{
    FScopeLock Lock(&SendMutex);
    SOCKET Sock = reinterpret_cast<SOCKET>(SocketHandle);
    packet.ownerId = MyOwner;
    SendAll((char*)&packet, sizeof(FVehiclePacket));
}
```

```
void UMyServer::ProcessVehiclePacket()
{
    FVehiclePacket data;
    int ProcessedCount = 0;
    while (ProcessedCount < MaxVehiclePacketsPerTick &&
        ClientThread->VehicleQueue->Dequeue(data))
    {
        ProcessedCount++;

        FVector Pos2 = FVector(data.X, data.Y, data.Z);
        FVector Dir2 = FVector(data.DirX, data.DirY, data.DirZ);

        if (APawnVehicle** CarPtr = SpawnVehicles.Find(data.carId))
        {
            (*CarPtr)->SetActorLocation(Pos2);
            FRotator Rot = Dir2.Rotation();
            (*CarPtr)->SetActorRotation(Rot);
        }
        else
        {
            UE_LOG(LogTemp, Error,
                TEXT("Vehicle not found carId=%d"),
                data.carId);
        }
    }
}
```

```
case EPacketType::Vehicle:
{
    FVehiclePacket* vehicle = reinterpret_cast<FVehiclePacket*>(PacketPtr);
    VehicleQueue->Enqueue(*vehicle);
    break;
}
```


핵심 시스템 - 서버와 통신

자주 수신되는 패킷(캐릭터 위치/방향 좌표)은 큐(Queue) 단위로 한 틱당 최대 처리 개수를 제한하였습니다.

도입 이유

데드락(Deadlock) 방지

- 한 틱 안에서 큐에 쌓인 패킷을 끝까지 다 처리하려고 하면, 처리 시간이 길어져 다른 작업(다른 큐 처리, 락 점유 등)과 충돌해 **데드락이 발생할 위험**이 있다고 생각하였습니다.
- 패킷 종류별로 처리 개수에 상한을 뒀서 특정 큐가 다른 리소스를 과도하게 오래 점유하지 않도록 하였습니다.

```
if (!GetWorld()->GetTimerManager().IsTimerActive(ServerTimer))
{
    GetWorld()->GetTimerManager().SetTimer(
        ServerTimer,
        this,
        &UMyServer::ProcessPacket,
        0.016f,
        true
    );
    UE_LOG(LogTemp, Warning,
        TEXT("SetTimer Complete"));
}
```

```
void UMyServer::ProcessPacket()
{
    PorcessCharacterPacket();
    ProcessSpawnAIPacket();
    ProcessDamagePacket();
    ProcessDeathPacket();
    ProcessConnectionPacket();
    ProcessItemPacket();
    ProcessVehiclePacket();
    ProcessRoomPacket(empty);
}
```

```
const int MaxCharacterPacketsPerTick = 50; //최대 50개의 캐릭터 패킷을 한 틱에 처리
const int MaxVehiclePacketsPerTick = 5; //최대 5개의 차량 패킷을 한 틱에 처리
```

```
void UMyServer::PorcessCharacterPacket()
{
    FCharacterPacket data;
    int ProcessedCount = 0;
    while (ProcessedCount < MaxCharacterPacketsPerTick && ClientThread->CharacterQueue->Dequeue(data))
    {
        ProcessedCount++;
    }
}
```

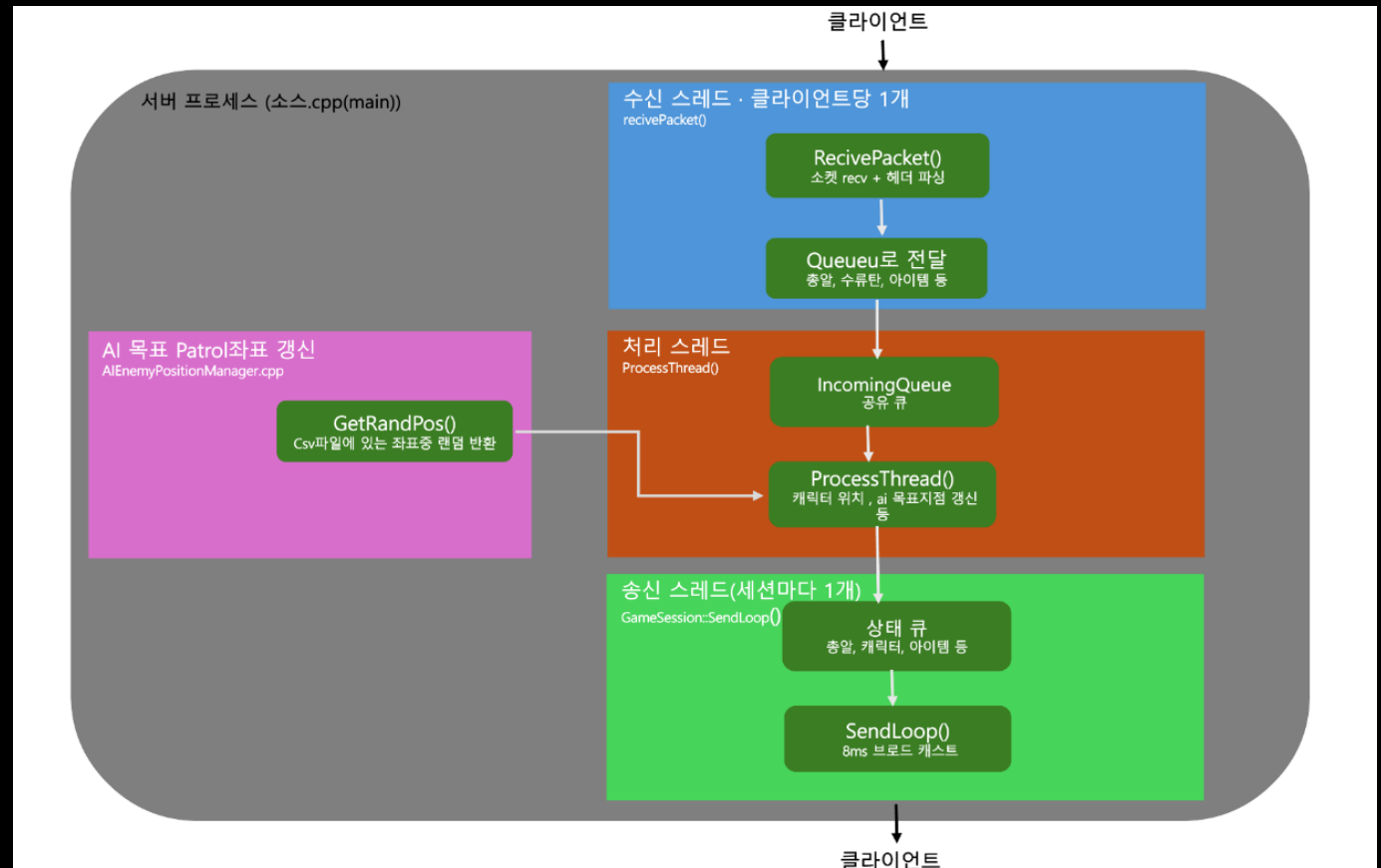
핵심 시스템 - 서버와 통신(서버)

Main.cpp(소스.cpp) : 클라이언트 연결을 받아 스레드를 띄우고,
패킷 헤더를 분석해 큐에 담는 클래스입니다.

GameSession.cpp : 큐에서 패킷을 꺼내 처리하고 SendLoop가
모든 클라이언트에 비동기 브로드 캐스트하도록합니다.

SessionManager.h : 방 생성/참가/시작 요청과 로비(대기실)
클라이언트 목록을 관리하는 싱글톤입니다.

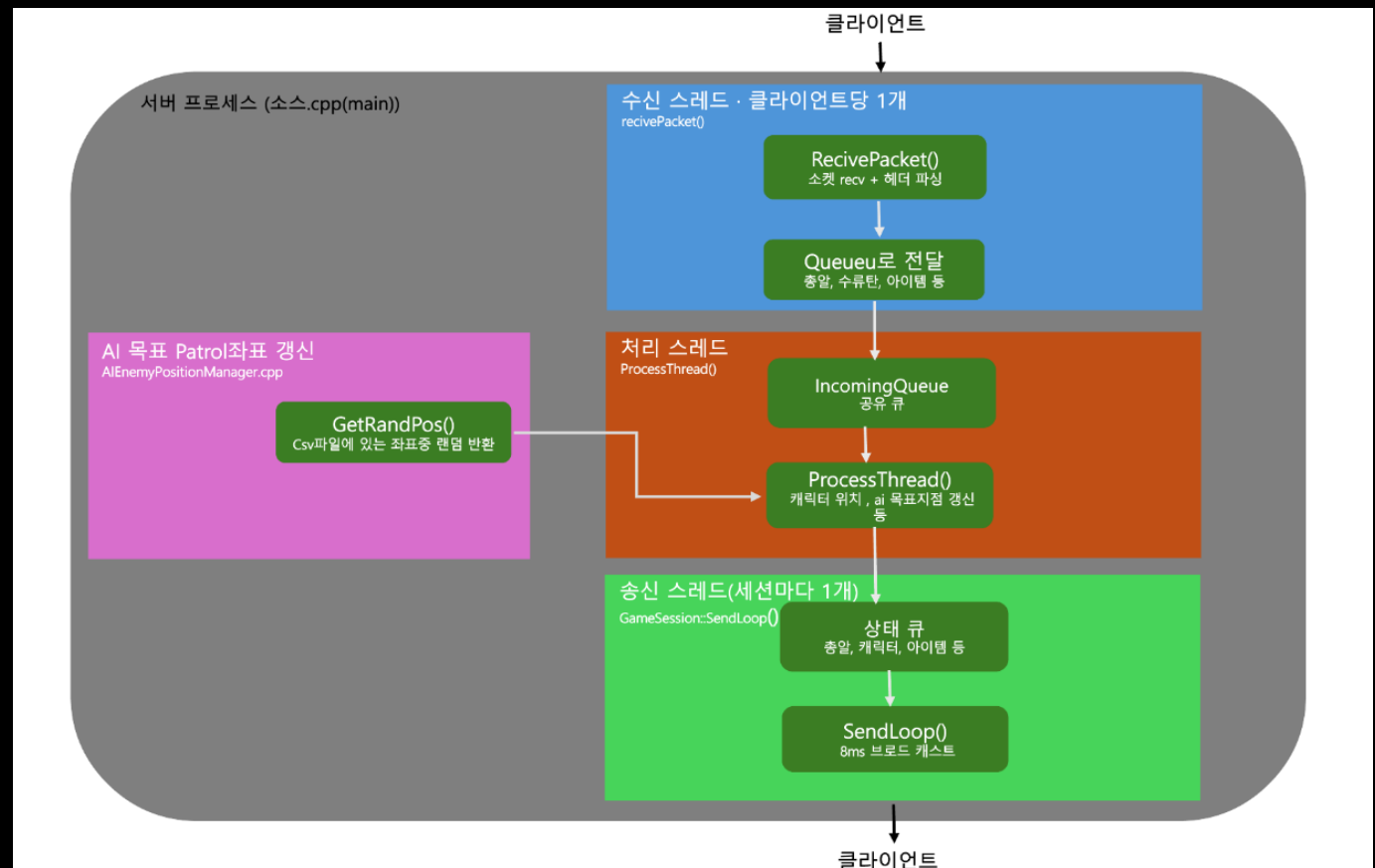
AIEnemyPositionManager.cpp : 사전 언리얼에서 생성해둔 임의의 좌표
CSV파일을 읽고 랜덤으로 임의의 위치를 반환해주는 클래스입니다.



-서버 구조도-

핵심 시스템 - 서버와 통신(서버)

서버는 1개(ProcessThread),
N개(클라이언트 수(여기서는 2개), 수신용),
1개(SendLoop, 송신용)으로
총 5개의 비동기 쓰레드로 구성됩니다.



-서버 구조도-

핵심 시스템 - 서버와 통신(서버)

ProcessThread()는 서버가 데이터를 처리하는 스레드로 모든 패킷이 담긴 공통 큐(IncomingQueue)에서 값을 꺼내 DataProcess()에서 분기를 나눠 처리하도록 합니다

```
void ProcessThread()
{
    while (true)
    {
        RecvPacket pkt;
        {
            std::lock_guard<std::mutex> lock(IncomingMutex);
            if (IncomingQueue.empty())
            {
                std::this_thread::sleep_for(std::chrono::milliseconds(1));
                continue;
            }
            pkt = std::move(IncomingQueue.front());
            IncomingQueue.pop();
        }

        FPacketHeader* header =
            reinterpret_cast<FPacketHeader*>(pkt.data.data());

        pkt.session->DataProcess(header, pkt.sock);
    }
}
```

핵심 시스템 - 서버와 통신(서버)

SendLoop()는 앞서 DataProcess에서 처리한 값을 클라로 보내는 역할을 합니다.

```
void GameSession::SendLoop()
{
    char SendBuff[sizeof(ServerBullet)];
    char SendBuff2[sizeof(FCharacterPacket)];
    char SendBuff3[sizeof(FGrenadePacket)];
    char SendBuff4[sizeof(FCharacterPacket)];
    char SendBuff5[sizeof(FInforamtionTextPacket)];
    char SendBuff6[sizeof(FDeathPacket)];
    char SendBuff7[sizeof(ServerBullet)];
    char SendBuff8[sizeof(FItemPacket)];
    char SendBuffVehicle[sizeof(FVehiclePacket)];

    while (true)
    {
        // 1. 캐릭터 수집
        std::vector<FCharacterPacket> SendList2;
```

```
        // 2. 총알
        while (!BulletQueue.empty())
        {
            ServerBullet pos;
            {
                std::lock_guard<std::mutex> lock(BulletMutex);
                if (BulletQueue.empty()) break;
                pos = BulletQueue.front(); BulletQueue.pop();
            }
            memcpy(SendBuff, &pos, sizeof(ServerBullet));
            for (int i = 0; i < clientSocketsIndex; i++)
            {
                if (pos.CharacterId == SocketToClientId[clientSockets[i]]) continue;
                std::lock_guard<std::mutex> lock(SendMutex);
                SendAll(clientSockets[i], SendBuff, sizeof(ServerBullet));
            }
        }
    }
}
```

핵심 시스템 - 서버와 통신(서버)

recvPacket()는 서버와 클라이언트가 연결될때마다 생성되는 쓰레드로 클라가 보내는 패킷을 읽어 간단한 패킷은 바로 처리하고 나머지는 IncomingQueue에 패킷을 넣는 역할을 합니다.

```
// 클라이언트가 방 생성/참가 패킷을 보내면 그때 세션에 등록됨
std::thread([ClientSocket]()
{
    recvPacket(ClientSocket, nullptr); // session은 처음엔 null
}).detach();
```

```
void recvPacket(SOCKET ClientSocket, std::shared_ptr<GameSession> session)
{
    FRecvBuffer RecvBuf;
    int lastPacketSize = 0, lastPacketType = -1;

    while (true)
    {
        int RecvLen = recv(ClientSocket,
            RecvBuf.GetWritePtr(),
            FRecvBuffer::BUFFER_SIZE - RecvBuf.WritePos, 0);

        if (RecvLen == 0)
        {
            printf("disconnected\n");
            return;
        }
        else if (RecvLen < 0)
        {
            int err = WSAGetLastError();
            if (err == WSAEWOULDBLOCK)
            {
                std::this_thread::sleep_for(std::chrono::milliseconds(1));
                continue;
            }
            printf("recv error: %d\n", err);
            return;
        }

        RecvBuf.OnWrite(RecvLen);

        while (true)
```

트러블 슈팅 - AI가 이동할 좌표 지정

대형 맵 환경에서 AI Patrol 좌표 설정의 어려움 (문제상황)

- 맵 규모가 커지면서 Nav Mesh Bounds Volume이 커버하는 영역도 광범위해집니다.
- 수동으로 AI의 순찰(Patrol) 좌표를 일일이 지정하기 비효율적이라고 생각했습니다.



트러블 슈팅 - AI가 이동할 좌표 지정

언리얼 에디터에서 NavMesh 폴리곤 데이터를

추출 → 서버에서 활용 (해결방안)

- 언리얼의 NavMesh(Recast 기반)는 Tile → Polygon → Vertex 구조로 이루어져있습니다.
- 폴리곤당 하나 당 Vertex좌표 중점을 계산해 CSV파일로 추출하여 서버에서 이 좌표를 읽어 랜덤 추출하는 방법을 생각해봤습니다.

```
//file->Poly->Verts
void AExportNavMeshVolumePos::ExportNavMeshPos()
{
    ARecastNavMesh* NavMesh = GetNavMeshData();
    if (!NavMesh) return;

    FString Out;

    int32 TileCount = NavMesh->GetNavMeshTilesCount();
    UE_LOG(LogTemp, Warning, TEXT("Tiles: %d"), TileCount);

    for (int32 TileIdx = 0; TileIdx < TileCount; ++TileIdx)
    {
        TArray<FNavPoly> Polys;
        if (!NavMesh->GetPolysInTile(TileIdx, Polys))
            continue;

        for (const FNavPoly& Poly : Polys)
        {
            TArray<FVector> Verts;
            if (!NavMesh->GetPolyVerts(Poly.Ref, Verts))
                continue;

            FVector Center(0);
            for (const FVector& V : Verts)
                Center += V;

            Center /= Verts.Num();

            Out += FString::Printf(TEXT("%f,%f,%f\n"),
                                    Center.X, Center.Y, Center.Z);
        }
    }

    FString Path = FPaths::ProjectDir() / TEXT("NavMeshCenters.csv");
    FFileHelper::SaveStringToFile(Out, *Path);
}
```



NavMeshCenters.csv × # styles.css

C:\Users> admin > Desktop > ss > fps > NavMeshCenters.csv

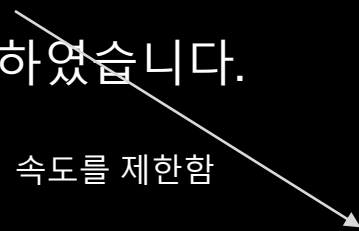
1	17290.000000,-40014.000000,-2567.569044
2	17271.000000,-39564.333333,12.430956
3	16865.666667,-39938.000000,12.430956
4	17342.250000,-40047.250000,12.430956
5	16834.000000,-39526.333333,32.430956
6	17290.000000,-41990.000000,-2567.569044
7	17290.000000,-41002.000000,-2567.569044
8	15314.000000,-43966.000000,-2567.569044
9	13338.000000,-43966.000000,-2567.569044
10	14326.000000,-43966.000000,-2567.569044
11	16302.000000,-40014.000000,-2567.569044
12	16302.000000,-39529.500000,32.430956
13	16302.000000,-40080.500000,12.430956
14	12350.000000,-42978.000000,-2567.569044
15	16302.000000,-41990.000000,-2567.569044
16	16302.000000,-41002.000000,-2567.569044
17	15314.000000,-42978.000000,-2567.569044
18	12350.000000,-41990.000000,-2567.569044
19	12350.000000,-41743.000000,892.430956

트러블 슈팅 - 서버 쓰레드

특정 시점에 클라이언트-서버 연결이 불규칙적으로 끊기는 현상 발생(문제상황)

- 평소엔 정상 동작하다가, 특정 조건에서 간헐적으로 클라이언트와 서버의 연결이 끊기는 현상이 발생하였습니다.
- 또한 일정속도 이상 주기로 패킷을 보내면 연결이 끊기는 현상 발생하였습니다.

속도를 제한함



```
FCharacterPacket man;
man.Header.Type = (int32)EPacketType::Character;
man.Header.Size = sizeof(FCharacterPacket);
man.CharacterId = -1; //쓰레기값
man.X = Location.X; //100.f;
man.Y = Location.Y; //200.f;
man.Z = Location.Z; //300.f;
man.DirX = ForwardDir.X; //1.f;
man.DirY = ForwardDir.Y; //0.f;
man.DirZ = ForwardDir.Z; //0.f;
man.Speed = Speed;
man.Sendtime = FPlatformTime::Seconds();
man.IsJump = bIsJumping;
man.IsFire = bIsFireing;
man.IsDeath = bIsDeath;
man.IsHeal = bIsHealKit;
man.IsHaveGun = bIsHaveGun;

/*
if (MyServer) {
    MyServer->MoveClient(man);
}*/

SendTimer += DeltaTime;
if (SendTimer >= 0.1f) // 20Hz
{
    if (MyServer) {
        MyServer->MoveClient(man);
    }
    SendTimer = 0;
}
```

플레이어 위치를 서버로 보내는 코드
(Tick 함수에 위치)

트러블 슈팅 - 서버 쓰레드

문제상황

수신 쓰레드와 데이터 처리 쓰레드를 분리(해결방안)

- 기존 구조: 패킷을 수신하는 쓰레드가 데이터 파싱/처리까지 함께 수행하고 있었습니다.
- 프로젝트 수행하면서 처리해야할 패킷이 증가 함에 따라 연산이 길어지고 다음 패킷 수신이 지연되었으며
- 이로 인해 수신 큐(Receive Queue)가 점점 쌓이고, 일정 수준을 넘으면 연결이 끊기는 것으로 추측했습니다.
- 개선: 수신 전담 쓰레드와 데이터 처리 전담 쓰레드를 별도로 생성하여 역할 분리 수신 쓰레드는 패킷을 받아 큐에 적재만 담당하도록 하였습니다.
- 처리 쓰레드는 큐에 쌓인 데이터를 별도로 가져와 헤더 검증, 패킷 처리를 하도록 하였습니다.

```
std::thread t(recvPacket, ClientSocket); // 쓰레드 생성
t.detach();
```



```
RecvBuf.OnWrite(RecvLen);
//printf("FPacketHeader before\n");
// 헤더도 못 읽으면 대기
if (RecvBuf.GetDataSize() < sizeof(FPacketHeader))
    continue;
// 헤더는 읽힘
if (RecvBuf.GetDataSize() >= sizeof(FPacketHeader))
{
    FPacketHeader* Header =
        reinterpret_cast<FPacketHeader*>(RecvBuf.GetReadPtr());
    //printf("GetDataSize before\n");

    // * 진행 중 패킷 정보 기록
    lastPacketSize = Header->Size;
    lastPacketType = Header->Type;

    int currentSize = RecvBuf.GetDataSize();

    // 패킷 전체가 아직 안 왔으면 대기
    if (RecvBuf.GetDataSize() < Header->Size)
    {
        printf("⌘ partial packet: need=%d, current=%d\n",
            Header->Size, currentSize);
        continue;
    }
    dataProcess(Header, ClientSocket);

    // 읽은 만큼 제거
    RecvBuf.OnRead(Header->Size);
}
```

끝까지 봐주셔서 감사합니다.